Neural Networks 185 (2025) 107200

Contents lists available at ScienceDirect

Neural Networks

journal homepage: www.elsevier.com/locate/neunet

Full Length Article

TDAG: A multi-agent framework based on dynamic Task Decomposition and Agent Generation

Yaoxiang Wang [a], Zhiyong Wu [b], Junfeng Yao [a], Jinsong Su [a,*]

[a] Xiamen University, Xiamen, 361005, China
[b] Shanghai AI Laboratory, Shanghai, China

ARTICLE INFO

Keywords:
Large Language Model
AI Agent
Travel Planning
Task Decomposition

ABSTRACT

The emergence of Large Language Models (LLMs) like ChatGPT has inspired the development of LLM-based agents capable of addressing complex, real-world tasks. However, these agents often struggle during task execution due to methodological constraints, such as error propagation and limited adaptability. To address this issue, we propose a multi-agent framework based on dynamic Task Decomposition and Agent Generation (TDAG). This framework dynamically decomposes complex tasks into smaller subtasks and assigns each to a specifically generated subagent, thereby enhancing adaptability in diverse and unpredictable real-world tasks. Simultaneously, existing benchmarks often lack the granularity needed to evaluate incremental progress in complex, multi-step tasks. In response, we introduce ItineraryBench in the context of travel planning, featuring interconnected, progressively complex tasks with a fine-grained evaluation system. ItineraryBench is designed to assess agents' abilities in memory, planning, and tool usage across tasks of varying complexity. Our experimental results reveal that TDAG significantly outperforms established baselines, showcasing its superior adaptability and context awareness in complex task scenarios.

1. Introduction

The emergence of Large Language Models (LLMs) (OpenAI, 2023; Zhang et al., 2022), such as ChatGPT, represents a noteworthy milestone in artificial intelligence, laying the groundwork for the creation of LLM-based agents (Nakajima, 2023; Significant-Gravitas, 2023) with the capacity to automate tasks on behalf of humans.

Despite advancements, LLM-based agents face substantial challenges in real-world tasks that demand complex planning, multi-stage reasoning, and tool utilization (Crispino et al., 2023; Qin et al., 2023). For instance, in a recent agent benchmark (Mialon et al., 2023), even GPT4 falls short with a success rate of 14%, while humans effortlessly exceed 92%. Empowering agents to effectively address real-world tasks is challenging, arising from difficulties in both benchmark and methodological perspectives.

From a benchmark perspective, we have witnessed lots of agent-related benchmarks in the passing year, covering domains like web manipulation (Yao et al., 2022), gaming (Wang et al., 2023), and software control (Li et al., 2023). However, the evaluation metrics in current benchmarks often lack the granularity needed to accurately reflect the incremental progress of agents. In complex multi-step tasks, even if an agent fails to achieve the ultimate targets, it might successfully accomplish some of the subtasks. It is essential to include these partial successes in evaluation metrics to faithfully capture the capabilities of agents. Reporting only a binary score (success or failure) can lead to misconceptions such as emergent abilities (Wei et al., 2022).

From a methodological perspective, existing approaches (Prasad et al., 2023; Sun et al., 2023a; Wang et al., 2023) aim to break down a complex request into a sequence of subtasks to reduce complexity and address the challenge of excessively long input. However, they encounter two notable drawbacks: error propagation and limited adaptability. In many studies, after the decomposition of the problem, all subtasks become fixed and unalterable. In such a setup, if an early subtask encounters failure, the error propagates, resulting in the failure of the entire task. Regarding subtask execution, prevailing efforts often involve the manual construction of hard-coded subagents. This static design tends to lack generality and is unable to scale effectively to handle diverse real-world tasks, given its labor and time-intensive nature.

To bridge the gap and facilitate research on complex reasoning and planning in LLM-based agents, we have made two contributions. First, we developed a benchmark that supports fine-grained evaluation of agents' progress. Concurrently, we developed a multi-agent framework for real-world problem-solving. This framework distinguishes

* Corresponding author.
E-mail addresses: wangyaoxiang@stu.xmu.edu.cn (Y. Wang), jssu@xmu.edu.cn (J. Su).

https://doi.org/10.1016/j.neunet.2025.107200
Received 6 June 2024; Received in revised form 4 September 2024; Accepted 19 January 2025
Available online 28 January 2025

itself through dynamic task decomposition and automatic subagent creation.

Our benchmark focuses on agent-assisted travel planning. Concretely, agents in our benchmark are required to act as personal assistants, leveraging computer tools like the databases and the code interpreter to execute tasks. To provide a holistic evaluation, our benchmark encompasses a spectrum of tasks, each varying in complexity. We also develop a simulator to mimic dynamic real-world scenarios, encompassing the entire pipeline of travel planning—from ticket booking to route/time planning. With the simulator, we are able to assess agents' partial task completion and deliver a nuanced score, which offers a more accurate and fair reflection of agents' capabilities.

Our multi-agent framework decomposes complex tasks into smaller, more manageable subtasks, while dynamically adjusting subsequent subtasks based on the completion status of preceding ones. For each subtask, a subtask-tailored subagent is generated using LLMs, which differs our design from conventional approaches that rely on a single agent or a set of manually designed subagents. Furthermore, these subagents are equipped with an evolving and adaptable skill library, designed to meet the challenges of varied and unpredictable environments.

Our contributions are summarized as follows:

- We develop ItineraryBench, featuring interconnected, progressively complex tasks and a fine-grained evaluation system.
- We introduce a multi-agent framework based on dynamic Task Decomposition and Agent Generation (TDAG), which decomposes a complex task into smaller subtasks, each managed by a specifically generated subagent.
- We conduct experiments on ItineraryBench, validating the effectiveness of the TDAG framework and highlighting the benefits of fine-grained evaluation.

2. Related work

2.1. Large language model-based agents

Large language models (LLMs) have demonstrated exceptional capabilities in language understanding, cognition, and reasoning (OpenAI, 2023; Ouyang et al., 2022; Zhang et al., 2022), inspiring the development of LLM-based agents (Park et al., 2023; Sun et al., 2023b; Wang et al., 2023). These agents integrate LLMs as their core processing units, enabling them to efficiently tackle complex tasks (Xi et al., 2023; Zhang et al., 2023). Their widespread application in various domains, from software development to scientific research, demonstrates their versatility (Boiko et al., 2023; Qian et al., 2023; Wu et al., 2024). This versatility is further exemplified by innovative open-source projects such as AutoGPT (Significant-Gravitas, 2023), BabyAGI (Nakajima, 2023), and Xagent (XAgent Team, 2023), which leverage LLMs for automated task execution by simply being provided with a task description and a set of available tools.

2.2. Enhancing agent planning with large language models

To enhance the reasoning and planning capabilities of LLM-based agents, various techniques like Chain-of-Thought prompting and task decomposition are employed. Typically, ReAct (Yao et al., 2023) uses an iterative process where a LLM generates thoughts and actions based on current observations until task completion. Reflexion (Shinn et al., 2023) introduces a self-reflection mechanism to improve reasoning quality. Meanwhile, ADAPT (Prasad et al., 2023) employs a recursive strategy that breaks down tasks into subtasks and allows further decomposition when necessary. We extend these capabilities further by introducing a multi-agent framework where the task is dynamically decomposed and each subtask is then assigned to specially generated subagent. This approach allows for more flexible and efficient task management, especially in complex and unpredictable scenarios.

2.3. Evaluating LLM-based agents

Recently, various benchmarks have been developed to evaluate the foundational abilities of these agents, such as decision-making, tool utilization, and embodied action capabilities (Liang et al., 2023; Lin et al., 2023; Qin et al., 2023; Xu et al., 2023). Additionally, there has been a focus on evaluating agents in specific real-world scenarios. For instance, AgentBench (Liu et al., 2023) has compiled a series of tasks set in multiple real-world scenarios to assess the performance of agents in completing a variety of tasks, including code, web (Yao et al., 2022), and game (Shridhar et al., 2021). Beyond scenario-specific evaluations, there is a growing interest in assessing the comprehensive abilities of agents. OpenAGI (Ge et al., 2023) is an open-source AGI research and development platform designed for solving multi-step, real-world tasks. GAIA (Mialon et al., 2023) introduces hundreds of real-world questions that necessitate fundamental abilities like reasoning, multi-modality handling. Different from existing studies, ItineraryBench uniquely assesses partial task completions, offering a more refined assessment framework for LLM-based agents. ItineraryBench focuses on travel planning, a scenario that offers a realistic and dynamic environment where tasks vary in complexity and require an agent to demonstrate abilities such as memory, planning, and tool usage, making it an ideal scenario to test agents' adaptability and incremental progress in completing complex, multi-step tasks.

3. Itinerarybench

In this section, we first provide an overview of ItineraryBench (Section 3.1), then explain the dataset construction (Section 3.2), discuss the multi-level evaluation metrics (Section 3.3), and highlight key features of our benchmark (Section 3.4).

3.1. Overview

We introduce ItineraryBench, a benchmark designed for evaluating agents in complex real-world scenarios, with a particular focus on travel planning. The core task for agents is to plan a trip that involves visits to multiple cities and attractions, while adhering to various constraints such as time and budget. This requires agents to interact with tools to access detailed information about transportation, accommodation, and attractions, as well as to facilitate planning.

As illustrated in Fig. 1, agents receive a task and are expected to produce a detailed itinerary that outlines activities for each segment of the trip. Formally, the itinerary comprises a sequence of predefined actions, including moving between places and cities, visiting spots, and staying in cities, as summarized in Table 1.

3.2. Dataset construction

We gathered information of 83 tourist attractions across 15 cities, 846 intra-city and 3903 inter-city transportation routes. Based on this foundation, we employed a combination of manual curation and code-assisted method to construct 364 distinct samples, exclusively reserved as the test set. Each sample represents a unique travel planning scenario, encompassing various combinations of cities, attractions, transportation modes, and scheduling constraints. Detailed statistics can be found in Appendix A.

Our benchmark encompasses three distinct task types, each anchored by a central storyline from which numerous data samples emerge. These types provide a comprehensive assessment of the agents' capabilities in handling varying complexities and real-world variables.

Type 1: Inter-city travel planning

The primary storyline of this type involves the user planning a journey that spans multiple cities, focusing on determining the order of cities to visit, the duration of stays, and the inter-city transportation. The final itinerary involves `'go_to_city'` and `'stay_in'` actions. There are variations in cities to visit, lengths of stay, and available activity hours, etc.

Table 1
Actions for itinerary construction.

Action	Description
go_to_place (origin, destination, depart_time, arrive_time)	Travel from one place to another within a city.
visit (place, begin_time, end_time)	Visit a tourist spot for a specified duration.
go_to_city (origin, destination, depart_time, arrive_time, ticket)	Travel from one city to another.
stay_in (city, begin_time, end_time)	Stay in a city for a specified duration.

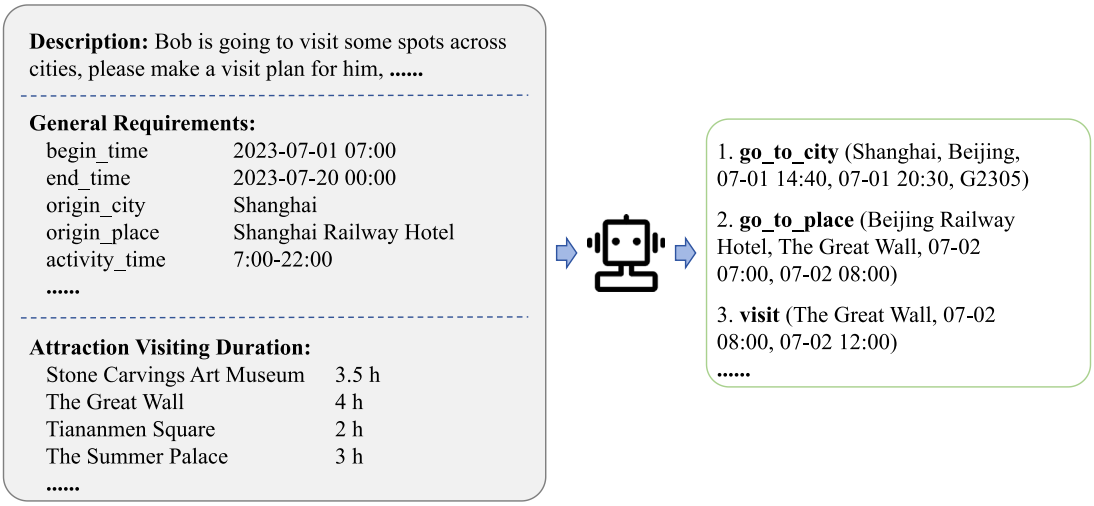

Fig. 1. Example of a travel planning task. The task specifies initial conditions and constraints for generating a travel itinerary.

Type 2: Intra-city travel planning

In this type of task, the user starts from a fixed location and plans to visit various attractions within a city. The final itinerary involves '`go_to_place`' and '`visit`' actions, incorporating constraints such as limited attraction opening hours and designated time allocations for meals and activities.

Type 3: Combined intra- and inter-city travel planning

This type of task is a composite of Types 1 and 2, involving planning visits to attractions within multiple cities. The itinerary involves '`go_to_city`', '`go_to_place`', and '`visit`' actions, representing a more complex version of travel planning.

Tools. To facilitate these tasks, agents have access to a database and a Python interpreter environment. The database contains information on transportation and attractions across cities and the Python interpreter can be used for calculation and numerical optimization

3.3. Evaluation metrics

To comprehensively evaluate the performance of agents, we develop a simulated environment that closely resembles real-world scenarios, assessing agents at three levels: Executability, Constraint Satisfaction, and Time and Cost Efficiency. Unlike traditional benchmarks that rely on a singular ground truth, we do not require a predefined ground truth. Instead, agents' plans are executed within the simulator, which provides real-time feedback by validating the correctness of the actions performed. This allows us to directly score the plans based on their performance within the environment, ensuring that multiple correct answers are accommodated.

Level 1 - Executability. Itineraries are evaluated on their basic feasibility. For example, '`go_to_city(Shanghai, Beijing, 07-01 14:40, 07-01 20:30, G2305)`' is assessed to ensure that the train's schedule matches the action's timing requirements. In the simulator, specific scoring items are designated to calculate the score. The scoring for Level 1 is calculated as follows:

$$s_1 = w_1 \times \frac{A_1}{B_1}, \tag{1}$$

where w_1 is the weight of Level 1 (60 points), A_1 is the number of successfully completed scoring items, and B_1 is the total number of scoring items in Level 1.

Level 2 - Constraint Satisfaction. This level assesses the itinerary's adherence to specified constraints like budget limits, stay durations, and opening hours of tourist spots. The scoring for Level 2 is calculated as follows:

$$s_2 = w_2 \times \frac{A_2}{B_2}, \tag{2}$$

where w_2 is the weight of Level 2 (20 points), A_2 is the number of successfully completed scoring items, and B_2 is the total number of scoring items in Level 2.

Level 3 - Time and Cost Efficiency. This level evaluates the itinerary based on its efficiency in terms of time and cost, while also considering the criteria from the previous levels. Efficiency is assessed against a predefined spending range $[a, b]$. A higher score is awarded for lower expenditures within this range, with a full score for expenses at or below the lower limit a, and no score for exceeding the upper limit b. The scoring formula is as follows:

$$s_3 = \begin{cases} w_3 & \text{if } s \le a \\ w_3 \times \left(1 - \frac{s-a}{b-a}\right) & \text{if } a < s < b \\ 0 & \text{if } s \ge b \end{cases}, \tag{3}$$

where w_3 is the weight of Level 3 (20 points) and s is the actual cost or time spent.

The values of a and b are determined by generating a large number of candidate itineraries and executing them within the simulator. This ensures that only the correct and executable itineraries are considered. From this pool, we randomly select 50 valid itineraries, and their time and cost values are used to calculate the mean and standard deviation. We set $a = \mu - \sigma$ and $b = \mu + \sigma$, where μ is the mean and σ is the standard deviation. This ensures that the range $[a, b]$ reflects realistic and achievable time and cost efficiency based on the performance of valid itineraries in the simulator.

These metrics provide a nuanced assessment of an agent's performance, especially in scenarios where only partial task completion

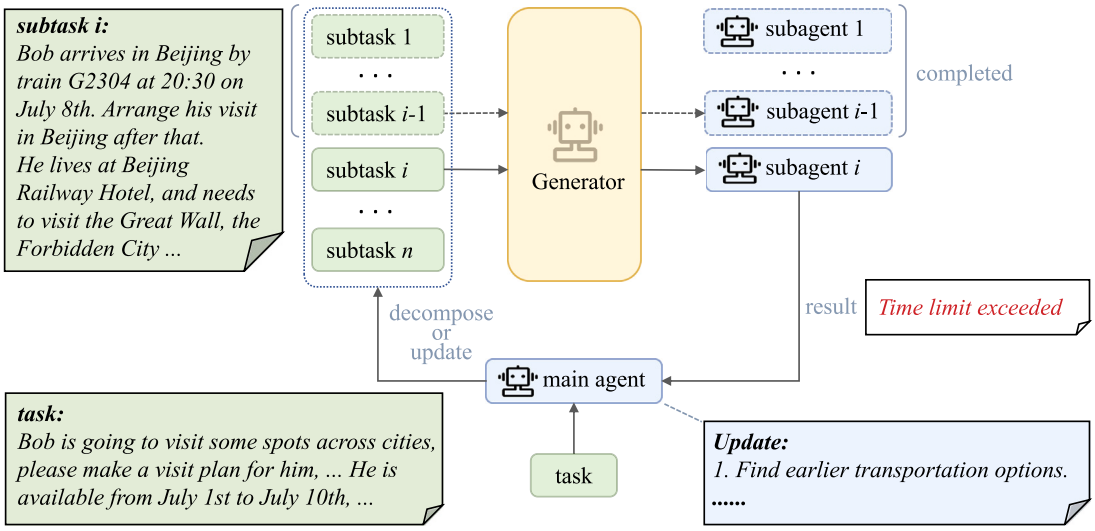

Fig. 2. Overview of TDAG framework. It shows the process of decomposing a complex task into multiple subtasks, which are then dynamically updated based on the completion status of preceding subtasks. Each subtask is assigned to a specially generated subagent, ensuring targeted and efficient task execution.

is achieved. This approach ensures a more accurate reflection of an agent's capabilities in handling complex and layered tasks.

Sequential Evaluation Process. It is important to note that the evaluation for higher levels only proceeds after the complete attainment of scores in the lower levels. For instance, an itinerary must first be feasible (Level 1) before it can be evaluated for compliance with specific constraints (Level 2) and its efficiency in terms of time and cost (Level 3) This approach reinforces the importance of a strong base in practical execution, as advanced optimizations are irrelevant if basic executability is not achieved.

3.4. Feature highlights

The features of ItineraryBench are summarized as follows:

1. **Multi-Faceted and Fine-Grained Evaluation.** Moving beyond binary success-failure assessments, our benchmark employs a nuanced scoring system, which recognizes varying degrees of task completion and evaluates agents at multiple levels, reflecting their overall performance in realistic applications.
2. **Interconnected Tasks and Progressive Complexity.** The tasks center around travel planning and are designed to increase in complexity through a series of evolving objectives and constraints. Each task is an extension of the previous one, simulating the interconnected nature of real-world problem-solving.
3. **Integration of Tools.** The benchmark incorporates tools like the Python interpreter and the database to evaluate agents' abilities to utilize tools and handle external information. The Python interpreter can be used for planning optimization and the database provides access to additional information, such as transportation and attraction details.

4. A multi-agent framework based on dynamic task decomposition and agent generation

In this section, we introduce a multi-agent framework based on dynamic Task Decomposition and Agent Generation (TDAG), as illustrated in Fig. 2. The framework is structured around two key components: (1) an advanced task decomposition strategy that dynamically adapts to the evolving requirements of complex tasks (Section 4.1), and (2) a flexible agent generation process that customizes agents for specific subtasks, enhancing adaptability and efficiency in varied scenarios (Section 4.2).

4.1. Dynamic task decomposition

In our framework, complex tasks are decomposed into smaller, more manageable subtasks, addressing the challenge of excessive irrelevant contexts that may hinder LLM performance.

The main agent decomposes the task, and each subagent is assigned a subtask, the process of which is represented as:

$$(t_1, t_2, \ldots, t_n) = \text{MainAgent}(T), \tag{4}$$

where t_i is the ith subtask and T is the original task.

The ith subtask is performed by the corresponding subagent$_i$, with the result as

$$r_i = \text{SubAgent}_i(t_1, t_2, \ldots, t_i, r_1, r_2, \ldots, r_{i-1}). \tag{5}$$

Note that each subagent focuses on a specific subtask, reducing the potential for noise of irrelevant information, thereby enhancing performance.

Crucially, these decomposed subtasks are not static, but will be dynamically adjusted based on the outcomes of preceding tasks, which is represented as:

$$t_i' = \text{Update}(t_i, r_1, r_2, \ldots, r_{i-1}), \tag{6}$$

where t_i' is the updated ith subtask, redefined based on the results $r_1, r_2, \ldots, r_{i-1}$ of preceding subtasks. This dynamic adjustment accounts for unexpected results (such as failures) or new information obtained during the execution of earlier subtasks, allowing for real-time recalibration and enhancement of the task-solving strategy.

4.2. Agent generation

Differing from previous methods (Prasad et al., 2023; Sun et al., 2023a), where tasks are assigned to predefined, fixed subagents, our framework allows for dynamic customization of subagents. This customization is achieved through LLM prompting, reflected in two aspects: generation of the tool document and an incremental skill library. By doing so, our framework significantly reduces the human effort required for agent setup and adaptation, making it more efficient in handling complex tasks.

4.2.1. Tool document generation

In standard tool documents, there are often issues of redundancy, unclear descriptions, which may mislead the agent into making inap-

Algorithm 1 Multi-Agent Framework Operation

Input: Task T, Tool document D, Skill library L
Output: Updated Skill library L, Task results R
$t\_list \leftarrow$ MainAgent.Decompose(T) ▷ Decompose task T
for t_i in t_list **do**
  $subagent_i \leftarrow$ AgentGenerator.Generate(D, L, t_i) ▷ Generate subagent
  $r_i \leftarrow subagent_i.Execute(t_1, t_2, \dots, t_{i-1}, t_i, r_1, r_2, \dots, r_{i-1})$ ▷ Execute subtask
  if t_i is successfully completed **then**
    $s \leftarrow subagent_i.SummarizeProcess()$ ▷ Summarize process
    $L \leftarrow$ Processor.UpdateSkillLibrary(L, s) ▷ Update Skill library
  end if
  $t\_list \leftarrow$ MainAgent.UpdateTasks(t_list, r_i) ▷ Update task list
end for
$R \leftarrow$ MainAgent.Submit($t_1, t_2, \dots, t_n, r_1, r_2, \dots, r_n$) ▷ Get the final result
return L, R ▷ Return updated library and results

propriate attempts to handle the task. Our framework involves generating an enhanced version of the original document by restructuring, clarifying and enriching the original content. By doing so, we aim to ensure that every piece of information presented to the agent is not only relevant but also optimally formatted for effective comprehension and application, thereby enhancing the agent's efficiency in task execution.

4.2.2. Incremental skill library

Our framework generates skills for subagents based on the agent behavior during task execution. These skills are then stored in a skill library, available for reference when agents face similar objectives in future tasks. The concept of a skill library is inspired by voyager (Wang et al., 2023), but with crucial adaptations. Unlike the environment of voyager, where the correctness of skills are directly verifiable through environmental feedback, our benchmark environment cannot guarantee the correctness of a skill. Therefore, the skills our agents develop are subject to continuous refinement in response to variable tasks and environmental conditions. When employing a skill, we use a small SentenceBERT model to retrieve the most relevant skill from the library based on the subtask's content. Once a subagent completes a subtask, it is prompted to summarize its process, which is then used to update the skill library. The format for skills is as follows:

- **Name:** The name of the skill.
- **Detail:** The details of the subtask.
- **Solution:** The solution for the subtask.

An agent dedicated to skill modification oversees the library, updating existing skills based on new data and experiences. This continuous cycle of generation, application, and refinement ensures that the skills in our library remain effective and adaptable to a wide range of scenarios. Algorithm 1 illustrates the step-by-step process of the overall framework.

5. Experiments

5.1. Settings

Our experiments are conducted across the three types of tasks of the Itinerary Bench as mentioned in Section 3. We employ the fine-grained evaluation system previously described, which allows scoring for partial completions of tasks. The baseline methodologies we have employed are:

- **ReAct** (Yao et al., 2023). A single agent iteratively completes the task, with each step involving the agent giving out thoughts and actions based on current observations.
- **P&S** (Wang et al., 2023). In this setting, the agent first creates a plan with multiple steps and then follows the ReAct pattern to solve the problem step by step.
- **P&E** (Sun et al., 2023a). The task is initially broken down into multiple subtasks, which are then sequentially assigned to a predefined executor. This decomposition strategy is commonly utilized in mainstream multi-agent frameworks such as MetaGPT (Hong et al., 2023) and AutoGen (Wu et al., 2023).
- **ADAPT** (Prasad et al., 2023). This method employs a recursive strategy to break down tasks into subtasks, which are then assigned to specific subagents. Subtasks can be further decomposed if not completed successfully.

To further explore the contributions of the dynamic decomposition and agent generation components in our TDAG framework, we conducted ablation studies by individually removing these features. Specifically, we evaluated the performance of our model without the dynamic decomposition process, using a static planner as in P&E method, and without the customization of subagents, relying instead on predefined subagents.

5.2. Main results

In our experiments, we use GPT-3.5-turbo-16k as the foundational model, and the results are shown in Table 2. Our method, TDAG, demonstrates superior performance compared to the established baselines. Furthermore, the ablation results confirm that the integration of both dynamic task decomposition and agent generation is crucial for optimizing task execution in complex, unpredictable environments.

Interestingly, the P&E approach, while allocating specific subtasks to designated subagents, does not outperform ReAct, where a single agent executes the entire task. This observation can be primarily attributed to the fixed nature of task assignments in P&E, as discussed in Section 5.3. Once subtasks are assigned, subagents lack the capacity to alter the plan, leading to challenges when initial subtasks do not produce expected results. This rigidity can make subsequent subtasks unexecutable, which is a critical limitation in dynamic and unpredictable environments. On the other hand, TDAG maintains the flexibility to adjust its execution strategy based on evolving task conditions, a key advantage in complex environments.

5.3. Error analysis

In our analysis, we identified and categorized the primary error types encountered by the different methods. These error types are as follows:

- **Cascading Task Failure (CTF)**: The entire task fails due to the failure of intermediate subtasks, often as a result of error propagation.
- **Commonsense Knowledge Error (CKE)**: Errors in basic understanding of real-world common sense, such as mismatch between the origin in `go_to_place` and the character's current location.

Table 2
Experimental results for each method on three task types of ItineraryBench. Numbers in bold indicate the highest score among all methods.

Method	Type 1	Type 2	Type 3	Avg.
ReAct	43.84	42.69	42.54	43.02
P&S	41.28	46.48	43.27	43.68
P&E	39.09	47.44	42.03	42.85
ADAPT	42.73	48.58	42.92	44.74
TDAG (Ours)	**49.78**	**50.96**	**46.51**	**49.08**
w/o Agent Generation	47.2	47.1	45.78	46.69
w/o Dynamic Decomposition	44.7	50.04	43.94	46.23

Table 3
Percentage of each type of errors encountered by different methods. The values in the table represent the percentage of each type of errors made by a method relative to the total number of that type of errors.

Method	CTF	CKE	EIM	CNC
ReAct	32.61	19.75	17.32	20.34
P&S	8.70	21.97	19.46	21.91
P&E	34.78	20.94	22.14	21.05
ADAPT	19.57	18.47	21.74	18.07
TDAG	4.35	18.87	19.33	18.63

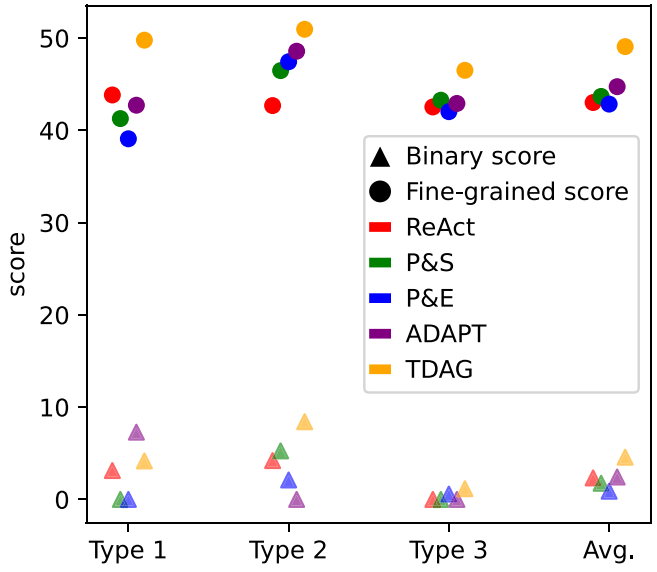

Fig. 3. Comparison of method performance using binary scoring and fine-grained evaluation.

- **External Information Misalignment (EIM)**: Errors in using external information, mainly stemming from the hallucination phenomenon (Ji et al., 2022), such as inconsistencies between database-provided ticket information and the travel plan.
- **Constraint Non-compliance (CNC)**: Errors related to not adhering to specific constraints of the task. This includes exceeding budget or time limits, etc.

As represented in Table 3, methods with fixed plans, such as P&E, are particularly prone to Cascading Task Failures, suggesting a lack of flexibility in adapting to unforeseen issues within a task. This type of error, where an early mistake can lead to the failure of an entire sequence of tasks, underscores the critical need for dynamic task decomposition. Regarding other types of errors, our analysis indicates that the TDAG method generally exhibits low error rates across CKE, EIM and CNC errors. This improvement is attributed to the ability of dynamically generated subagents to focus on specific subtasks, leveraging their tailored capabilities to navigate the intricacies of each challenge effectively. Notably, ReAct outperforms more complex approaches in minimizing EIM errors. A possible reason could be that complex methods generate excessive context, which may increase the likelihood of hallucination phenomena.

Table 4
Experimental Results for WebShop and TextCraft. Following Prasad et al. (2023), we report the reward score and success rate (%) for WebShop and success rate for TextCraft.

Method	WebShop		TextCraft
	Reward score	Success rate	Success rate
ReAct	42.1	29.0	19.0
P&S	58.2	23.0	18.5
P&E	27.7	17.0	27.0
ADAPT	60.0	44.0	52.0
TDAG	**64.5**	**45.0**	**73.5**

5.4. Binary scoring v.s. Fine-grained evaluation

To highlight the significance of the fine-grained evaluation, we conducted an experiment comparing it with the traditional binary scoring method. Given the complexity of tasks, a full score in Level 1 - Executability was used as the criterion for successful task completion, with the results presented in Fig. 3. It reveals that in challenging tasks with a low probability of completion, binary scoring fails to distinguish between different methods effectively. On the other hand, the fine-grained evaluation method provides a more detailed insight, reflecting the extent of task completion even when perfect execution is not achieved.

5.5. Generalizability

To further assess the generalizability of our TDAG method, we conducted additional experiments on the following datasets:

- **WebShop** (Yao et al., 2022). Developed as a simulated e-commerce environment, WebShop requires agents to navigate through various webpage types, reformulate queries, comprehend and interact with text-heavy webpages. Following Shinn et al. (2023), we report performance on 100 user instructions.
- **TextCraft** (Prasad et al., 2023). As a text-only environment mirroring the crafting component of Minecraft, TextCraft presents tasks with a natural compositional structure akin . Agents in TextCraft aim to craft target items using available resources and a set of defined actions. The test split contains 200 samples.

Following Prasad et al. (2023), we employed GPT-3.5-turbo for WebShop and gpt-3.5-turbo-instruct for TextCraft. Due to the lack of tool document and the monotonous nature of the tasks, we did not engage in agent generation. Instead, we utilized the predefined agents from ADAPT.

The experimental results, as illustrated in Table 4, demonstrate that our method outperforms the baseline methods in both reward score and success rate metrics. The results reinforce our assertion that TDAG is not only effective in complex, multifaceted tasks but also excels in more monotonous and straightforward scenarios.

6. Conclusion

In this paper, we first present ItineraryBench, a benchmark for evaluating LLM-based agents in complex, real-world tasks, particularly in travel planning. It stands out with its interconnected, progressively challenging tasks and a nuanced evaluation system that goes beyond binary scoring. This approach allows for a more precise assessment of an agent's capabilities, especially in partial task completions. Additionally, we introduce the TDAG framework, enhancing adaptability and success in diverse tasks by dynamically decomposing complex tasks into manageable subtasks, each handled by a custom-generated subagent. Our experimental results show that TDAG significantly outperforms existing baselines across several benchmarks.

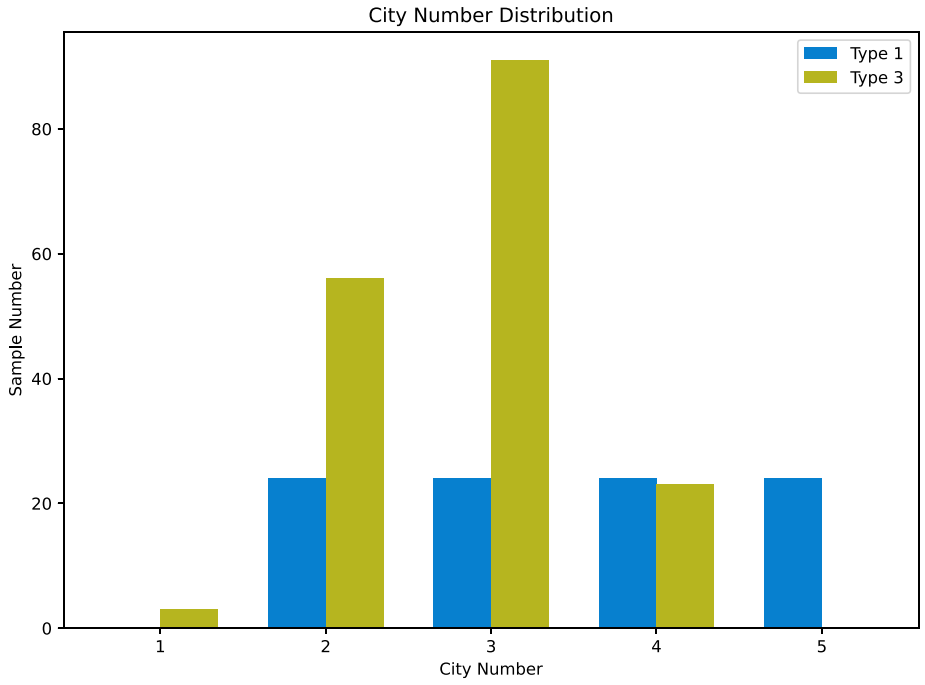

Fig. 4. City Number Distribution. The horizontal axis in the figure represents the number of cities, and the vertical axis represents the number of tasks that contain a certain number of cities.

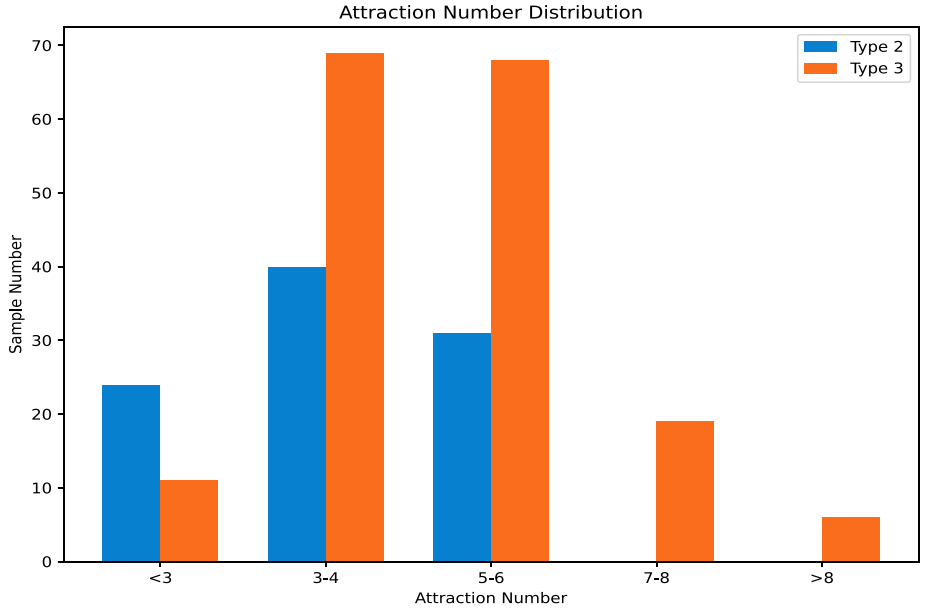

Fig. 5. Attraction Number Distribution. The horizontal axis in the figure represents the number of attractions, and the vertical axis represents the number of tasks that contain a certain number of attractions.

CRediT authorship contribution statement

Yaoxiang Wang: Writing – original draft, Methodology, Data curation, Conceptualization. **Zhiyong Wu:** Writing – review & editing, Project administration, Conceptualization. **Junfeng Yao:** Writing – review & editing. **Jinsong Su:** Writing – review & editing, Project administration.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The project was supported by National Natural Science Foundation of China (No. 62036004, No. 62276219), Natural Science Foundation of Fujian Province of China (No. 2024J011001), and the Public Technology Service Platform Project of Xiamen (No. 3502Z20231043). We also thank the reviewers for their insightful comments.

Appendix A. Benchmark details

In this appendix, we provide detailed insights into the composition and characteristics of the ItineraryBench, focusing on the distribution

System prompt: You are an autonomous intelligent agent tasked with making travel plans for Bob. To be successful, it is very important to follow the following rules:

- You should only issue one action at a time.
- You should reason step by step and then issue the next action.
- Your response should be formatted as follows:
  THOUGHT: The thought process to achieve the goal, including the subtask to be handled.
  ACTION: The action you call to assign the subtask or submit the task.

EXTERNAL RESOURCES:

- A database containing information about train tickets, attractions, and city transportation.
- A Python notebook to execute Python code for numerical operations and planning.

PLAN AND SUBTASK: The task needs to be broken down into multiple subtasks and handed over to other subagents to complete. You may need to wait for results from other agents before proceeding to the next step of the task. If you need help from other agents, please clearly describe the task objectives, background, and precautions of the subtask.

```
{
    "subtask_name": string,
    "goal": string,
    "criticism": string,
    "milestones": list[string],
    "result_format": optional
}
```

Important Notice:

- Always make feasible and efficient plans that can lead to successful task solving. Never create new subtasks that are similar or the same as the existing subtasks.
- For subtasks with similar goals, try to do them together in one subtask with a list of subgoals, rather than splitting them into multiple subtasks.
- You only need to complete the tasks in the given requirement, don't do anything extra.
- You can plan multiple subtasks if you want.

Your Workflow:

- You will first be given a task.
- Then you will make a plan and start to solve it step by step. Multiple subtasks need to be completed during the solution process. For each subtask, use `<action>subagent_handle(subtask_name)</action>` and wait for the response of other agents. After getting a response, continue with the task.
- If you encounter a failure in the execution of a subtask, adjust the plan and corresponding subtasks as necessary before continuing with the task.
- Finally, call `over()` to indicate task completion. After the task requirements are completed, call `over()` immediately.

Available Actions:

- `<action>subagent_handle(subtask_name)</action>`: If you want to assign a subtask to other agents for completion, call `<action>subagent_handle(subtask_name)</action>`. Before you call it, provide the subtask surrounded by `<subtask></subtask>`.
- `<action>over()</action>`: When you think the task is completed, call "`<action>over()</action>`". After the content in the task requirements is completed, call `over()` immediately without making unnecessary planning.

Now you will be given a task. Start planning and give me an action.

Fig. 6. Prompt used by the MainAgent for managing the entire task decomposition and execution flow in the travel planning task.

Table 5
Distribution of task types in ItineraryBench.

Type	Number
Type 1	96
Type 2	95
Type 3	173

of task types, city and attraction number distributions, and the various constraints that define the travel planning scenarios.

A.1. Task type distribution

Table 5 presents the distribution of task types within the benchmark. The tasks are categorized into three types, each representing a distinct travel planning scenario. Type 1 tasks involve inter-city travel planning, focusing on arranging city visits and transportation. Type 2 tasks are centered around intra-city navigation, requiring planning visits to various attractions within a single city. Type 3 combines elements of both Type 1 and Type 2, presenting a more complex scenario of navigating through multiple cities and their attractions. The

System prompt: You are a prompt generator, responsible for generating prompts for an autonomous agent. You will be given a subtask that the agent needs to complete. The subtask is part of a larger task. Your goal is to optimize the relevant action documents to ensure the agent can perform the required actions effectively. The agent uses actions strictly according to the provided document, so the optimized document will ensure that the actions are carried out correctly.
The original document is as follows:

```
<original Action Document>{ORIGINAL_DOCUMENT}</original Action Document>
```

*** Important Notice ***

- Surround the new document with <action document></action document>.
- You don't need to complete the task, but refine the document for the subtask at hand.
- Optimize the document by combining the main task and subtasks, ensuring only the necessary actions for the subtask are included.
- Ensure no critical details are omitted to avoid execution errors.

Your Workflow:

- You will receive a subtask that requires refinement of the action document.
- Based on the subtask, refine the original document by optimizing only the necessary actions for completion.
- After refining the document, output it using the <action document></action document> format.

Fig. 7. Prompt used for refining tool documents to ensure accurate execution.

System prompt: You are tasked with analyzing two tasks and their respective solutions:
Task 1: task1
Solution 1: solution1
Task 2: task2
Solution 2: solution2
Your task is to analyze both solutions and determine the following:

- Assess whether the solution for Task 2 (Solution 2) can be applied to or is relevant for solving Task 1.
- Provide a detailed analysis of the compatibility and effectiveness of Solution 2 in the context of Task 1.

If you find that Solution 2 can assist in solving Task 1, update Solution 1 accordingly by incorporating elements from Solution 2. Ensure that the updated solution is practical, concise, and addresses potential issues that may arise from this integration.
If Solution 1 should be updated, respond with:

```
<action>update("solution_content")</action>
```

If no update is necessary, respond with:

```
<action>keep()</action>
```

Fig. 8. Prompt used for skill updates, ensuring that the agent efficiently merges relevant solutions while maintaining simplicity and clarity.

Table 6
Types of constraints in travel planning.

Constraint	Description
Time limit	The travel must be completed within a certain period.
Budget	A budget for the trip.
Transportation	Includes both intra-city and inter-city transportation.
City duration	A specific duration of stay in a city.
Spot duration	A specific duration of stay at a tourist spot.
Specific hotel	Staying in a specified hotel.
Activity time	Designated times when the user is allowed to engage in activities.
Spot opening hours	The operating hours of tourist attractions.
Rest time	Designated times when the user must rest.

distribution of these task types is designed to ensure a comprehensive evaluation of agents' capabilities across different planning complexities.

A.2. City and attraction distribution

The distributions of cities and attractions within the dataset are visualized in Figs. 4 and 5, respectively. These distributions highlight the benchmark's diversity and the range of scenarios that agents must navigate. The city distribution figure shows the number of times each city appears across the dataset, reflecting the geographical spread and frequency of visits required in various tasks. Similarly, the attraction distribution figure illustrates the variety and frequency of tourist spots that must be considered in the travel plans.

System prompt: You are an autonomous agent specialized in learning from environmental feedback and following rules to perform correct and efficient actions. Your decisions must always be made independently without seeking user assistance.
You can take actions to interact with the environment to obtain additional information. All your actions should be surrounded with <action> and </action>.
To be successful, it is very important to follow the following rules:

1. You should only issue one action at a time.
2. You should reason step by step and then issue the next action.
3. Your response should be formatted as follows:
   THOUGHT: the thought process to achieve the goal.
   ACTION: the action you call to get information or submit the task.

Your Workflow:

1. You will first be given a task.
2. Handle the task until you need to use an action, call this action and wait for the result. Give your thought and action in every response.
3. Finally, call <action>over()</action> to submit the sub-task and give detailed suggestions for future planning in the thought. Surround the result of the task and the suggestions with <result> and </result>. If there is no result that perfectly meets the requirements, return a result that relatively meets the requirements. Do not return an empty result.

Available Actions:
{{tool document}}
<action>over()</action>
When you think the task is completed, call <action>over()</action>.
Only these actions can be called. Other actions that you can complete yourself can be performed directly in the THOUGHT step by step.
Important Rules

- You must follow your workflow.
- You are more than a Large Language Model (LLM); you have the capability to do actual tasks rather than simply give guidance or write text.
- Submit the task immediately by calling <action>over()</action> if no further actions are needed.

Plan Overview
The total task is:
{{total task}}
The part that has been completed is:
{{completed task}}
The subtask you need to complete is:
{{current task}}

Important Notice

- You can at most use 5 steps of action calls. After that, you must use over() and submit the sub-task.
- Always try to solve the given task by exploring possible solutions and using tools efficiently.
- If you encounter the same error twice, try another way to solve the problem.
- If there is no result that perfectly meets the requirements, return a result that relatively meets the requirements. Do not return an empty result.

Fig. 9. Prompt used by sub-agents for handling subtasks, ensuring independent decision-making and efficient task completion while following structured rules and workflows.

A.3. Constraints in travel planning

Table 6 outlines the various constraints incorporated into the travel planning tasks. These constraints simulate real-world conditions and challenges, requiring agents to optimize their plans while adhering to limitations such as time, budget, transportation options, and specific requirements for city and spot durations. Including constraints such as specific hotel stays, designated activity times, and rest periods further enhances the benchmark's realism, pushing the boundaries of agents' problem-solving abilities.

It is noteworthy that all attraction and transportation information were either obtained from public websites or self-edited, ensuring that there are no issues related to privacy, copyright, or ethics.

Appendix B. Experimental details

B.1. Implementation of the incremental skill library

The Incremental Skill Library includes skill generation, storage, retrieval and update. Skills are organized into three fields: 'Name', 'Detail', and 'Solution'. After completing a subtask, subagents use LLM prompting to summarize their solution as a skill. The storage and update process is detailed in Algorithm 2. To maintain uniqueness, we check for redundancy using the SentenceBERT model 'all-mpnet-base-v2' to generate embeddings for the skill details. A similarity threshold, $\theta = 0.7$, is set to identify potential duplicates. A new skill is considered redundant and excluded from the library if there are already k or more skills in the library with a similarity greater than θ, where

Algorithm 2 Skill Storage and Update in Incremental Skill Library

Input: New skill s_{new}, Skill library L, Similarity threshold θ, Similarity limit k
Output: Updated Skill library L
$embedding_{new} \leftarrow$ SentenceBERT.GenerateEmbedding($s_{new}.Detail$) ▷ Generate embedding for the new skill
$similar\_count \leftarrow 0$ ▷ Initialize similarity counter
for s_i in L **do**
  $embedding_i \leftarrow$ SentenceBERT.GenerateEmbedding($s_i.Detail$) ▷ Generate embedding for existing skill
  $similarity \leftarrow$ ComputeCosineSimilarity($embedding_{new}, embedding_i$) ▷ Compute similarity
  if $similarity > \theta$ **then**
    $similar\_count \leftarrow similar\_count + 1$ ▷ Increment similarity count
    if $similar\_count \geq k$ **then**
      $L \leftarrow$ LLM.UpdateSkills(L, s_{new}) ▷ Update the existing skills in the library
      return L ▷ Return updated library
    end if
  end if
end for
$L \leftarrow L \cup s_{new}$ ▷ Add new skill to the library if not redundant
return L ▷ Return updated library

$k = 2$. However, instead of exclusion, when the maximum number of similar skills is reached, the LLM automatically update the existing skills according to the redundant one. For retrieving skills, the same SentenceBERT model is employed. When a subagent is assigned a new subtask, the model finds the top k most relevant skills by comparing the embeddings of the subtask's name to those in the library. These skills are then provided to the subagent in textual form, aiding in the execution of the task.

B.2. Models and hyperparameters

For the experiments on ItineraryBench, we deploy the gpt-3.5-turbo-16k. Given the complex nature and the lengthy process required to solve tasks in this benchmark, methods lacking a task decomposition strategy, such as ReAct, require long context length to achieve task completion. For WebShop and TextCraft, we adhere to the configurations recommended by Prasad et al. (2023), utilizing the gpt-3.5-turbo and gpt-3.5-turbo-instruct respectively. To ensure experimental consistency and replicability, we set the temperature to 0.

B.3. Related prompts

As shown in Fig. 6, our main agent is responsible for dynamically decomposing tasks, assigning subtasks to specialized sub-agents, and controlling the overall task flow. The main agent follows a structured workflow, ensuring that each subtask is executed step by step, with adjustments made as necessary based on the success or failure of individual subtasks. This approach allows for flexible task management and dynamic adaptation to the challenges posed by complex, multi-step tasks. Fig. 7 shows the prompt used for document refinement. This prompt ensures that the agent refines and optimizes the tool documents related to each subtask, allowing the agent to accurately perform the required actions without unnecessary steps or omissions. In addition, Fig. 8 shows the prompt used for skill updates, which allows the agent to analyze and compare solutions from different tasks and determine whether any enhancements or updates should be made to the existing skill set based on the new information. Fig. 9 shows the prompt used by sub-agents to handle and complete their assigned subtasks. This prompt guides the sub-agent through its workflow, ensuring that it takes appropriate actions and submits the task results with consideration for the task's success and future planning.

Data availability

Data will be made available on request.

References

Boiko, D. A., MacKnight, R., & Gomes, G. (2023). Emergent autonomous scientific research capabilities of large language models. arxiv e-prints arXiv:2304.05332.

Crispino, N., Montgomery, K., Zeng, F., Song, D., & Wang, C. (2023). Agent instructs large language models to be general zero-shot reasoners. arXiv e-prints arXiv:2310.03710.

Ge, Y., Hua, W., & Mei (2023). OpenAGI: When LLM meets domain experts. *NeurIPS 2023*, http://dx.doi.org/10.48550/arXiv.2304.04370.

Hong, S., Zhuge, M., Chen, J., Zheng, X., Cheng, Y., Zhang, C., Wang, J., Wang, Z., Yau, S. K. S., Lin, Z., Zhou, L., Ran, C., Xiao, L., Wu, C., & Schmidhuber, J. (2023). MetaGPT: Meta programming for a multi-agent collaborative framework. arXiv:2308.00352.

Ji, Z., Lee, N., Frieske, R., Yu, T., & Su (2022). Survey of Hallucination in natural language generation. arXiv e-prints arXiv:2202.03629.

Li, H., Su, J., Chen, Y., Li, Q., & Zhang, Z. (2023). SheetCopilot: Bringing software productivity to the next level through large language models. *NeurIPS 2023*, http://dx.doi.org/10.48550/arXiv.2305.19308.

Liang, Y., Zhu, L., & Yang, Y. (2023). Tachikuma: Understading complex interactions with multi-character and novel objects by large language models. arXiv e-prints arXiv:2307.12573.

Lin, J., Tomlin, N., Andreas, J., & Eisner, J. (2023). Decision-oriented dialogue for human-AI collaboration. arxiv e-prints arXiv:2305.20076.

Liu, X., Yu, H., Zhang, H., Xu, Y., Lei, X., Lai, H., Gu, Y., & Ding (2023). AgentBench: Evaluating LLMs as agents. arxiv e-prints arXiv:2308.03688.

Mialon, G., Fourrier, C., Swift, C., Wolf, T., LeCun, Y., & Scialom, T. (2023). GAIA: a benchmark for general AI assistants. arxiv e-prints arXiv:2311.12983.

Nakajima, Y. (2023). Babyagi. URL https://github.com/yoheinakajima/babyagi.

OpenAI (2023). GPT-4 technical report. arxiv e-prints arXiv:2303.08774.

Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., & Agarwal (2022). Training language models to follow instructions with human feedback. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, & A. Oh (Eds.), *35, Advances in neural information processing systems* (pp. 27730–27744). Curran Associates, Inc..

Park, J. S., OBrien, J. C., Cai, C. J., Ringel Morris, M., Liang, P., & Bernstein, M. S. (2023). Generative agents: Interactive simulacra of human behavior. arXiv e-prints arXiv:2304.03442.

Prasad, A., Koller, A., Hartmann, M., Clark, P., Sabharwal, A., Bansal, M., & Khot, T. (2023). Adapt: As-needed decomposition and planning with language models. arXiv e-prints arXiv:2311.05772.

Qian, C., Cong, X., Liu, W., Yang, C., Chen, W., & Su (2023). Communicative agents for software development. arXiv e-prints arXiv:2307.07924.

Qin, Y., Hu, S., Lin, Y., & Chen (2023). Tool learning with foundation models. arXiv preprint arXiv:2304.08354.

Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language agents with verbal reinforcement learning. *NeurIPS 2023*, http://dx.doi.org/10.48550/arXiv.2303.11366.

Shridhar, M., Yuan, X., Côté, M.-A., Bisk, Y., Trischler, A., & Hausknecht, M. (2021). ALFWorld: Aligning Text and Embodied Environments for Interactive Learning. In *ICLR*.

Significant-Gravitas (2023). Auto-GPT: An autonomous GPT-4 experiment. https://github.com/Significant-Gravitas/Auto-GPT.

Sun, S., Liu, Y., Wang, S., Zhu, C., & Iyyer, M. (2023). PEARL: Prompting large language models to plan and execute actions over long documents. arXiv e-prints arXiv:2305.14564.

Sun, Q., Yin, Z., Li, X., Wu, Z., Qiu, X., & Kong, L. (2023). Corex: Pushing the boundaries of complex reasoning through multi-model collaboration. arXiv:2310.00280.

Wang, G., Xie, Y., Jiang, Y., Mandlekar, A., Xiao, C., Zhu, Y., Fan, L., & Anandkumar, A. (2023). Voyager: An open-ended embodied agent with large language models. In *Intrinsically-motivated and open-ended learning workshop @neurIPS2023*.

Wang, L., Xu, W., Lan, Y., Hu, Z., Lan, Y., Lee, R. K.-W., & Lim, E.-P. (2023). Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. ACL 2023.

Wei, J., Tay, Y., Bommasani, R., Raffel, C., Zoph, B., & Borgeaud (2022). Emergent abilities of large language models. arXiv e-prints arXiv:2206.07682.

Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., Awadallah, A. H., White, R. W., Burger, D., & Wang, C. (2023). AutoGen: Enabling next-gen LLM applications via multi-agent conversation framework. arXiv:2308.08155.

Wu, Z., Han, C., Ding, Z., Weng, Z., Liu, Z., Yao, S., Yu, T., & Kong, L. (2024). OS-copilot: Towards generalist computer agents with self-improvement. arXiv e-prints arXiv:2402.07456.

XAgent Team (2023). XAgent: An autonomous agent for complex task solving.

Xi, Z., Chen, W., Guo, X., He, W., Ding, Y., & Hong (2023). The rise and potential of large language model based agents: A survey. arXiv e-prints arXiv:2309.07864.

Xu, B., Liu, X., Shen, H., & Han (2023). Gentopia: A collaborative platform for tool-augmented LLMs. arXiv e-prints arXiv:2308.04030.

Yao, S., Chen, H., Yang, J., & Narasimhan, K. (2022). WebShop: Towards scalable real-world web interaction with grounded language agents. arXiv e-prints arXiv:2207.01206.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. In *ICLR 2023*.

Zhang, S., Roller, S., Goyal, N., & Artetxe (2022). OPT: Open pre-trained transformer language models. arXiv e-prints arXiv:2205.01068.

Zhang, Z., Yao, Y., Zhang, A., Tang, X., & Ma (2023). Igniting language intelligence: The Hitchhiker's guide from chain-of-thought reasoning to language agents. arXiv e-prints arXiv:2311.11797.